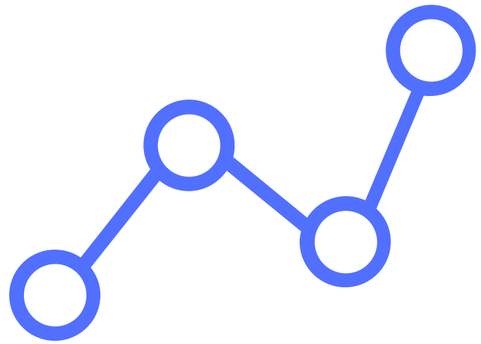
VIBE CODING 기초

7차시

클라우드 & 배포 입문

클라우드 플랫폼 · 기술 스택 · 배포 · 미니 프로젝트 기획

해달 교육운영진 허진수 | 문의: 단톡방



INDEX

7차시 수업 순서 (110분)

1 클라우드 플랫폼 개요

2 프론트엔드 · 백엔드와 배포

3 미니 프로젝트 기획

4 마무리 & 질의응답

오늘의 시간표

파트	시간	내용
① 클라우드	19:00 ~19:30	클라우드 개념, 서비스 모델, GCP 서비스 → 🍵 10분 휴식
② 스택 · 배포	19:40 ~20:10	프론트엔드 · 백엔드 기술 스택, 배포 방법 → 🍵 10분 휴식
③ 프로젝트	20:20 ~20:40	미니 프로젝트 주제 소개, 서비스 기획 실습
④ 마무리	20:40 ~20:50	질의응답, 8차시(구현 · 발표) 안내

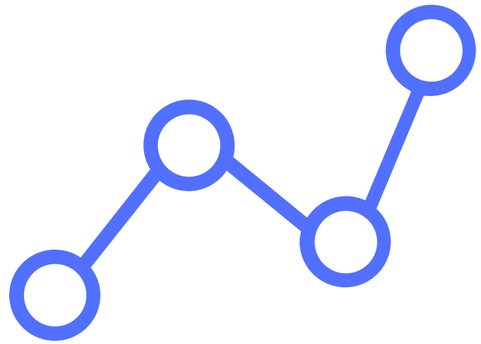
PART 01

클라우드 플랫폼

클라우드 개념 · 서비스 모델 · Google Cloud

⏱️ 교육 시간 30분

해달 교육운영진 허진수 | 문의: 단톡방



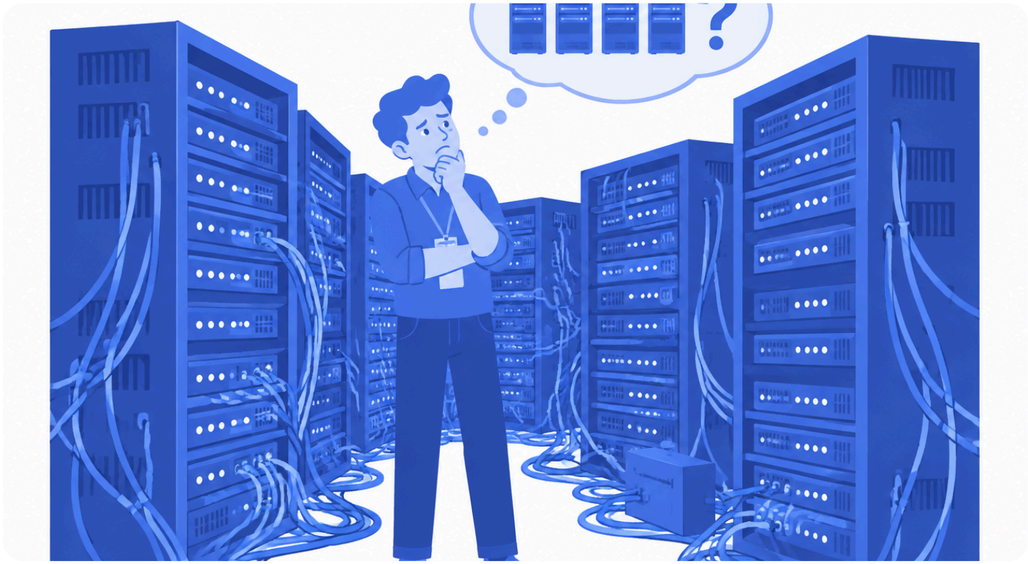
클라우드 컴퓨팅이란?

인터넷으로 빌려 쓰는 컴퓨터

① 서버 · 저장소 · DB를 인터넷으로 빌리고
② 필요할 때 바로 늘리거나 줄이고
③ 쓴 만큼만 요금을 내는 방식

- 예: Google Cloud · AWS · Azure
- 핵심 단어: 온디맨드(On-demand)
- 정의 출처: 미국 NIST (2009)

클라우드 이전 vs 이후



클라우드 이전

1. 서버를 미리 사서 직접 설치
2. '몇 대가 필요할까?' 추측
3. 남거나 모자라면 손해

클라우드 이후

1. 필요할 때 클릭 몇 번
2. 몇 초 만에 만들고 지우기
3. 쓴 만큼만 결제

On-premise vs Cloud

구분	On-premise (직접 보유)	Cloud (빌려 쓰기)
초기 비용	**서버 구매비가 큼**	거의 없음! 쓴 만큼만 결제
준비 기간	**주문 · 설치에 수 주**	클릭 몇 번이면 수 분 안에 준비!
확장	**장비를 더 사야 함**	설정만 바꾸면 자동으로 늘어남
관리	**전산실에서 직접 관리**	하드웨어 관리는 클라우드 회사 몫!

클라우드 핵심 개념 4가지

온디맨드 (On-demand)

- ✓ 필요할 때 바로 생성
- ✓ 몇 초 만에 서버 준비 완료

종량제 (Pay-as-you-go)

- ✓ 쓴 만큼만 요금 지불
- ✓ 초 단위 청구도 가능

핵심 개념

탄력성 (Auto Scaling)

- ✓ 사용자가 늘면 자동 확장
- ✓ 줄어들면 자동 축소

리전 · 영역 (Region · Zone)

- ✓ 전 세계 데이터센터 위치
- ✓ 사용자와 가까운 곳 선택

클라우드의 장점과 단점

장점 ① 비용 절감

- ✓ 초기 투자 없이 시작
- ✓ 규모의 경제로 더 저렴하게

단점 ① 요금 폭탄

- ✓ 설정 실수 = 과금 위험
- ✓ 예산 알림 설정은 필수!

장점 vs 단점

장점 ② 확장성 · 자동화

- ✓ 트래픽에 맞춰 자동 확장
- ✓ 보안 패치 · 백업 자동화

단점 ② 종속 · 의존

- ✓ 한 회사에 묶임 (Lock-in)
- ✓ 인터넷이 끊기면 사용 불가

가상화 = 클라우드의 기반

없는 것을 있는 것처럼

① 물리 서버 1대 위에
② 하이퍼바이저(Hypervisor)를 깔고
③ 가상 머신(VM) 여러 대를 동시에 실행

- VM마다 다른 OS 설치 가능
- 남는 CPU · 메모리를 알뜰하게
- 클라우드 서버 = 대부분 VM

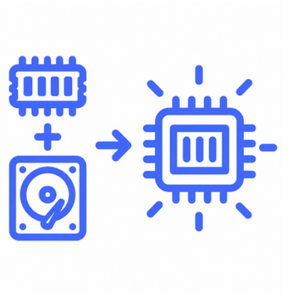
가상화 3가지 방식

자원 분할
Partitioning

서버 1대를
여러 대처럼
(서버 가상화)

자원 통합
Clustering

서버 여러 대를
1대처럼
(클러스터링)



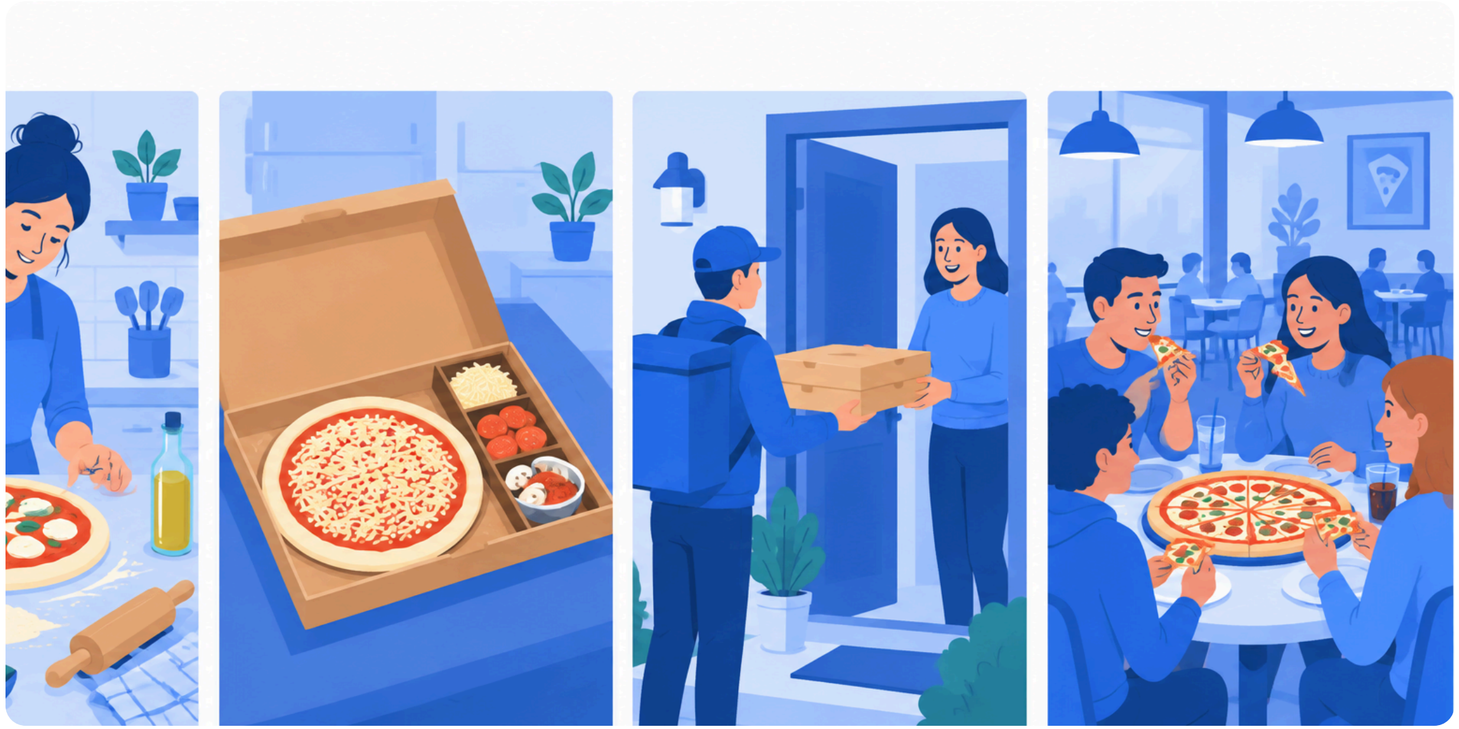
자원 가상화
Emulation

메모리 + 디스크를
더 큰 메모리처럼
(가상 메모리)

책임 모델의 진화

모델	내가 관리하는 것	대표 예시
On-premise	하드웨어부터 전부	회사 전산실 서버
IaaS	OS · 런타임 · 앱 · 데이터	Compute Engine · AWS EC2
PaaS	앱 코드 · 데이터만	Cloud Run · App Engine · Vercel
SaaS	그냥 사용 (설정 정도)	Gmail · Notion · Canva

피자로 이해하는 IaaS · PaaS · SaaS



직접 → 밀키트 → 배달 → 외식

On-premise → IaaS → PaaS → SaaS

오른쪽으로 갈수록 내가 할 일이 줄어요

Google Cloud (GCP)

구글의 클라우드

① 검색 · YouTube와 같은 인프라
② 전 세계 40개+ 리전 (서울 포함)
③ 신규 가입 시 무료 크레딧

- AWS · Azure와 3대 클라우드
- AI(Gemini · Vertex AI)에 강점
- 무료 등급부터 체험 가능

Google Cloud 대표 서비스

컴퓨팅 (Compute)

- ✓ Compute Engine · GKE
- ✓ Cloud Run · Cloud Functions

스토리지 · DB

- ✓ Cloud Storage (파일 저장)
- ✓ Cloud SQL · Firestore (DB)

GCP 4분류

빅데이터 (Data)

- ✓ BigQuery (대용량 분석)
- ✓ Pub/Sub (메시지 전달)

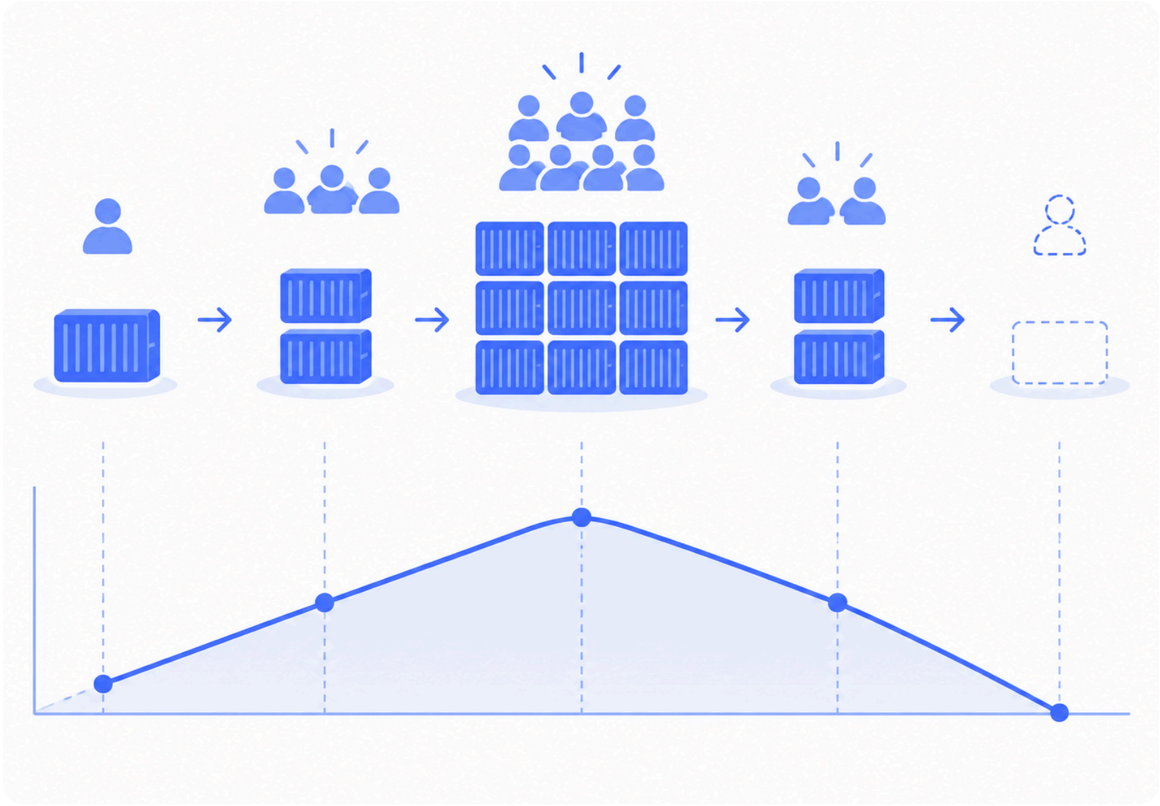
AI · 머신러닝

- ✓ Vertex AI (모델 학습 · 배포)
- ✓ Gemini API (생성형 AI)

컴퓨팅 서비스 고르기

서비스	한 줄 비유	이럴 때 쓰세요
Compute Engine	컴퓨터 한 대 통째로	OS까지 직접 다루고 싶을 때
GKE	컨테이너 여러 개 관리	큰 서비스를 여러 조각으로 운영할 때
Cloud Run	컨테이너 하나 맡기기	웹 서버 · API를 간단히 올릴 때 (추천!)
Cloud Functions	함수 하나만 실행	이벤트에 반응하는 작은 작업

Cloud Run (서버리스 컨테이너)



코드만 올리면 끝

① 코드를 컨테이너로 포장하고

② Cloud Run에 올리면

③ 접속량에 맞춰 자동으로 늘고 줄어요

- 요청 없으면 0개 → 비용 0원
- 서버 관리 · OS 패치 불필요
- HTTPS 주소 자동 발급

GCP vs AWS 서비스 비교

역할	AWS	GCP
가상 머신	EC2	Compute Engine
서버리스 컨테이너	App Runner	Cloud Run
서버리스 함수	Lambda	Cloud Functions
파일 저장소	S3	Cloud Storage

거버넌스 뼈대: 리소스 계층

계층 구조

- Organization → Folder → Project → Resource 순서로 관리!
- 권한은 위에서 아래로 상속(Inheritance)돼요.

단계별 역할

Organization	Folder	Project
회사 전체 (최상위 루트)	부서 · 환경 단위 (개발 / 운영)	과금 · 권한의 기본 단위 리소스는 꼭 여기 안에!

IAM: 누가 · 무엇을 · 어디에

IAM 정책

누구에게(Who) 어떤 리소스에(Where)
어떤 역할(Role)을 줄지 정하는 규칙이에요.

✔ WHO

누가 (계정)

사용자 · 그룹
서비스 계정(프로그램용)

✔ WHAT

무엇을 (역할)

권한을 모아 둔 세트
예: 뷰어 · 편집자

✔ WHERE

어디에 (리소스)

프로젝트 · VM
저장소 버킷 등

쉬는 시간

10분 쉬어 갈게요!

19:40에 다시 시작합니다

다음 파트: 프론트엔드 · 백엔드 기술 스택과 배포
잠깐 스트레칭하고 오세요!

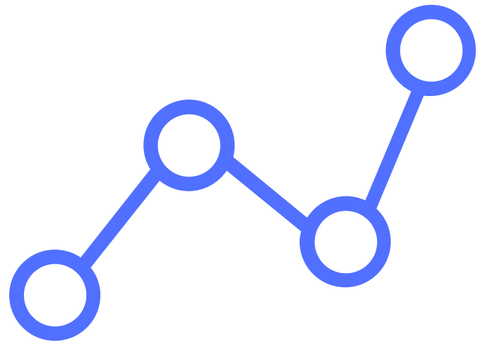
PART 02

기술 스택 · 배포

프론트엔드 · 백엔드 · 배포 서비스 · 배포 방법

⏱️ 교육 시간 30분

해달 교육운영진 허진수 | 문의: 단톡방



식당으로 이해하는 웹 서비스

웹 서비스 = 식당

① 홀 = 프론트엔드 (손님이 보는 화면)

② 주방 = 백엔드 (주문을 처리)

③ 창고 = 데이터베이스 (재료 보관)

- 손님(사용자)은 홀만 봐요
- 주문(요청) → 주방 → 음식(응답)
- API = 홀과 주방 사이의 주문서

프론트엔드란?

📁 브라우저가 받는 3가지 파일

index.html → 뼈대 (무엇이 있나)

```
<button>신청하기</button>
```

style.css → 옷 (어떻게 보이나)

```
button { color: blue; }
```

app.js → 동작 (누르면 어떻게)

```
button.onclick = () =>
  alert('신청 완료!');
```

눈에 보이는 화면

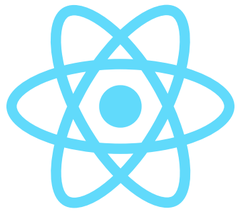
① 사용자가 보고 누르는 모든 것

② 브라우저 안에서 실행돼요

③ 기본 재료: HTML · CSS · JS

- 식당으로 치면 '홀'
- 1차시 자기소개 페이지도 프론트엔드!
- 요즘은 프레임워크로 빠르게

프론트엔드 기술 스택



React
(리액트)

점유율 1위

컴포넌트로 화면 조립

자료 · 생태계 최대

Next.js
(넥스트)

React + 서버 기능

검색 노출(SEO)에 강함

Vercel이 만든 프레임워크

Vue
(뷰)

배우기 쉬운 문법

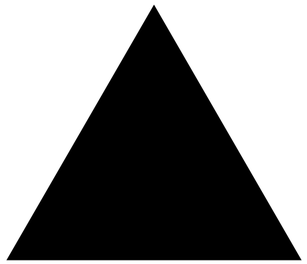
HTML과 비슷한 구조

가볍게 시작하기 좋음

프론트엔드 스택 장단점

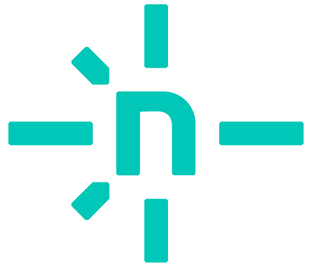
기술	장점	단점
React	자료 1위	선택지가 너무 많아 처음엔 헷갈려요
Next.js	SEO 강함	서버 / 클라이언트 개념이 어려워요
Vue	쉬운 문법	국내 채용 시장이 작은 편이에요
Svelte	가볍고 빠름	자료 · 라이브러리가 아직 적어요

프론트엔드 배포 서비스



Vercel

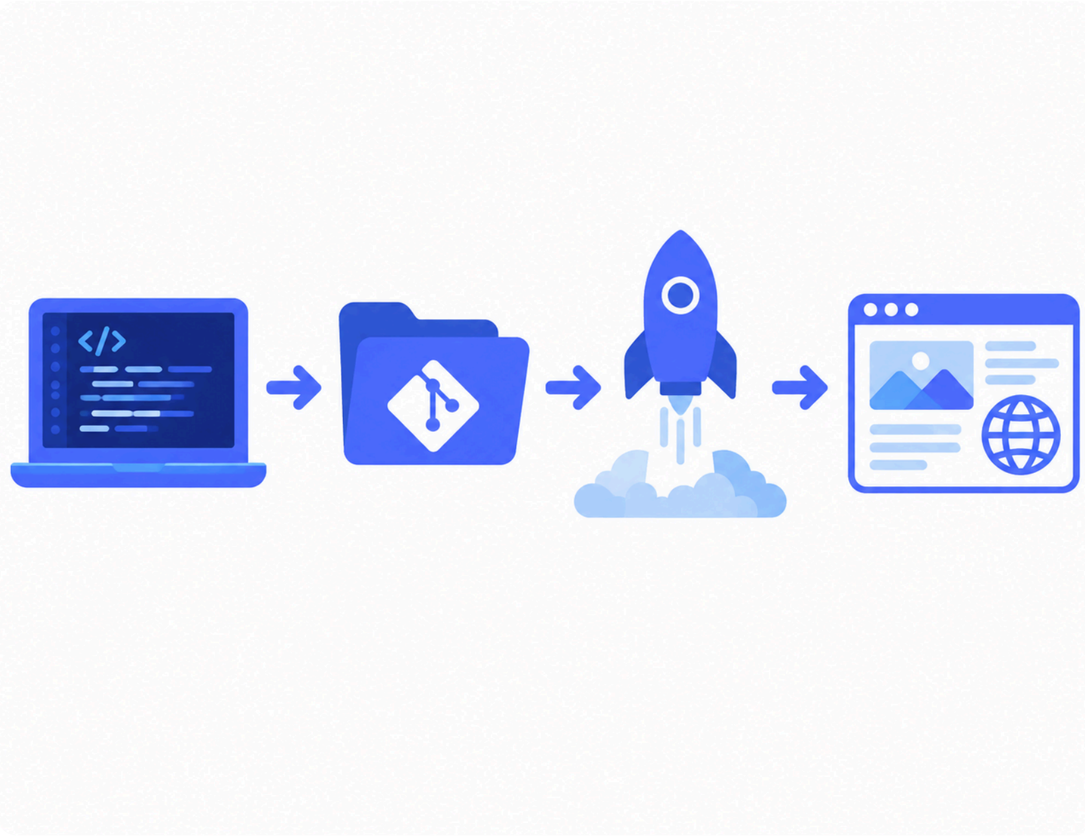
Next.js 제작사 · Git 연동 자동 배포
개인 프로젝트는 무료(Hobby)



Netlify

드래그 앤 드롭으로도 배포
무료 플랜 제공

프론트엔드 배포 방법 (Vercel)



GitHub → Vercel → 주소 발급

① GitHub에 코드 올리기 (push)
② vercel.com → Add New Project
③ 저장소 Import → Deploy 클릭
④ OOO.vercel.app 주소 완성!

백엔드란?

🐍 main.py (FastAPI 예시)

```
from fastapi import FastAPI

app = FastAPI()

# 요청: GET /events
@app.get("/events")
def get_events():
    # DB에서 행사 목록을 꺼내서
    return [
        {"name": "신입 환영회"},
        {"name": "해커톤"}
    ]  # JSON으로 응답!
```

보이지 않는 주방

① 화면의 요청(Request)을 받고

② 데이터를 처리 · 저장한 뒤

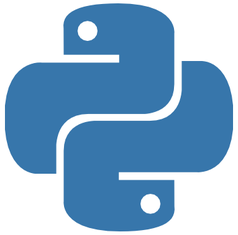
③ 결과를 응답(Response)

이 약속된 통로 = API

백엔드 기술 스택

Node.js
Express · NestJS

JS 하나로 풀스택
실시간 채팅에 강함



Python
FastAPI · Django

AI · 데이터에 강함
문법이 쉬움

Java
Spring Boot

대기업 · 금융권
대규모 서비스

백엔드 스택 장단점

고르는 기준

이미 익숙한 언어가 정답이에요!
바이브코딩이라면 자료가 많은 스택이 더 유리합니다.

✔ STACK 01

Node.js

👍 JS 하나로 프론트+백
👎 무거운 계산엔 약함

✔ STACK 02

Python

👍 AI · 데이터에 강함
👎 실행 속도는 느린 편

✔ STACK 03

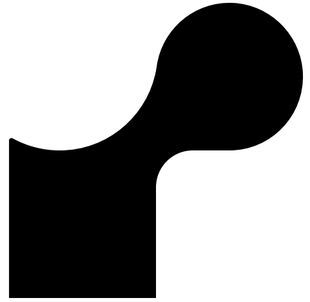
Java

👍 대규모 · 안정성
👎 코드 양이 많음

백엔드 배포 서비스

Railway

GitHub 연결만 하면 자동 배포
체험 크레딧 후 사용량 과금



Render

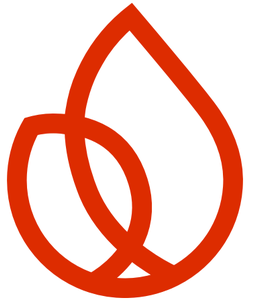
무료 플랜 제공
15분 미사용 시 잠듦(슬립)

BaaS: 백엔드를 빌려 쓰기

Supabase

PostgreSQL DB + 로그인 기능

오픈소스 · 무료 플랜

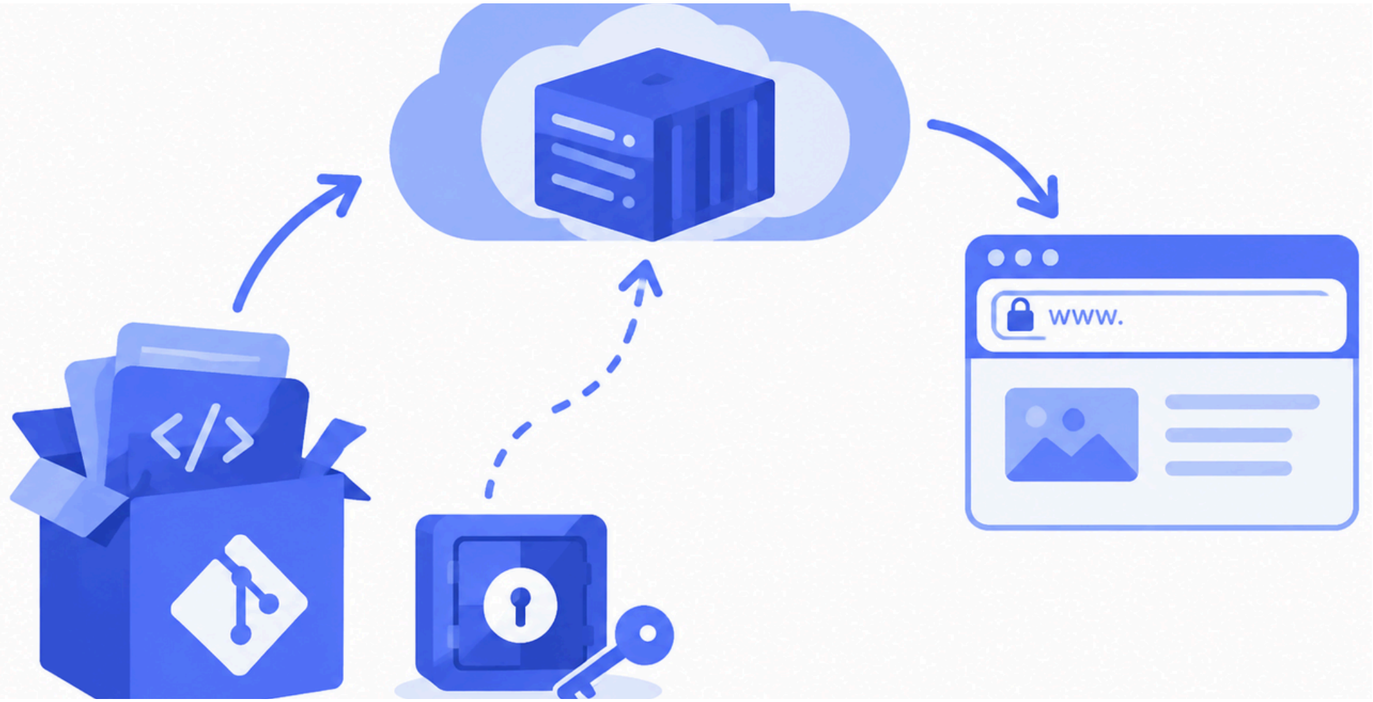


Firebase

구글의 BaaS · 실시간 DB

로그인 · 호스팅까지 한 번에

백엔드 배포 방법 (Railway)



배포 4단계

① GitHub에 코드 올리기 (push)
② New Project → Deploy from GitHub
③ Variables에 API 키 등록
④ Generate Domain → 주소 발급

- .env 파일은 GitHub에 올리지 않기
- 무료 크레딧 · 사용량 확인
- 에러는 Logs 탭에서 확인

배포 전 체크 3가지

배포 전에 꼭!

내 컴퓨터에서 되던 게 서버에서 안 되는 경우가 많아요.
아래 세 가지만 확인하면 대부분 해결됩니다!

CHECK 01

API 키 숨기기

.env는 GitHub에 올리지 않기
키는 환경 변수로 등록

CHECK 02

무료 한도 확인

예산 알림 · 사용량 확인
안 쓰는 서버는 바로 삭제

CHECK 03

로컬에서 먼저 실행

내 PC에서 실행 · 빌드 확인
에러는 배포 로그에서 확인

쉬는 시간

수고했어요! 10분 휴식

20:20에 다시 시작합니다

다음 파트: 미니 프로젝트 기획
동아리에서 불편했던 점을 떠올려 보세요!

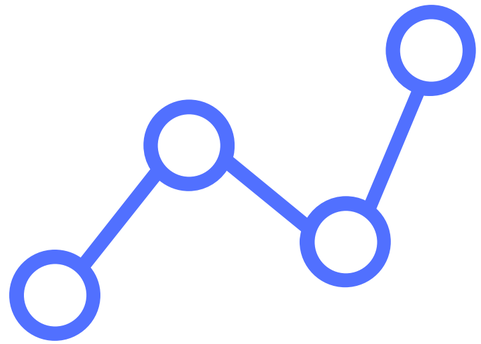
PART 03

미니 프로젝트 기획

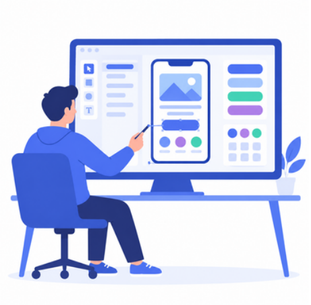
개발 역할 · 프로젝트 주제 · 1페이지 기획서

⏱️ 교육 시간 20분

해달 교육운영진 허진수 | 문의: 단톡방



여러분이 원하는 개발 역할은?



프론트엔드
화면 담당

눈에 보이는 결과가
좋다면!

백엔드
서버 · 데이터 담당

데이터 흐름을
고민하는 게 재밌다면!

풀스택
화면 + 서버 + 기획

처음부터 끝까지
다 만들고 싶다면!

미니 프로젝트 주제

우리의 주제

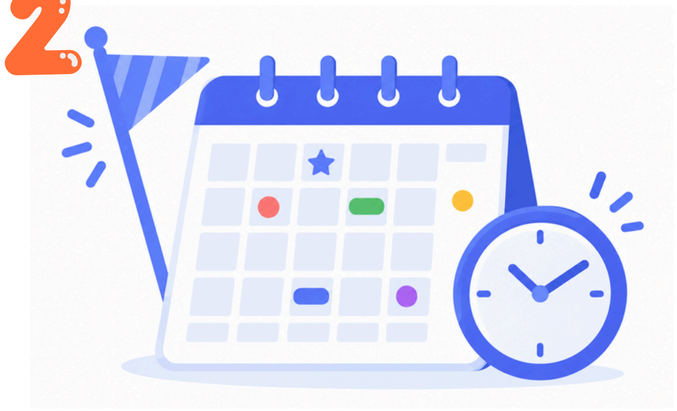
“ 신입 회원의 입장에서
동아리에 필요한 서비스
만들기! ”

세부 주제 예시

1

동아리 FAQ 챗봇

2

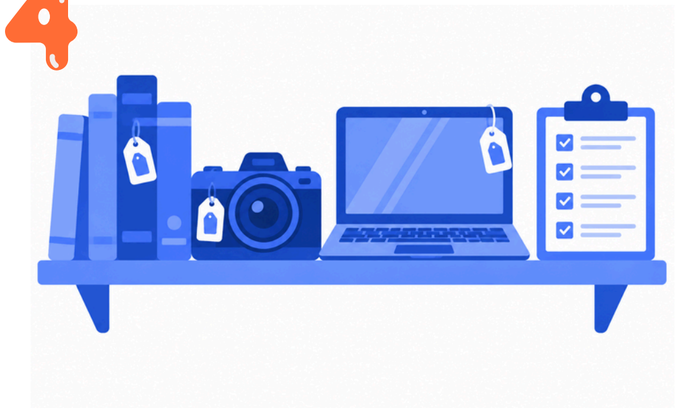


행사 일정 캘린더

3

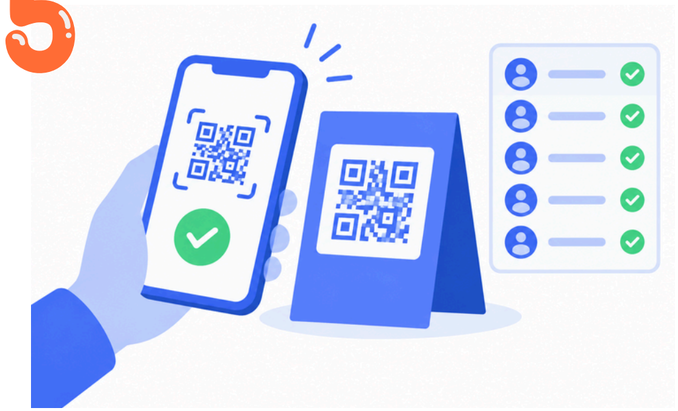
스터디 매칭 보드

4



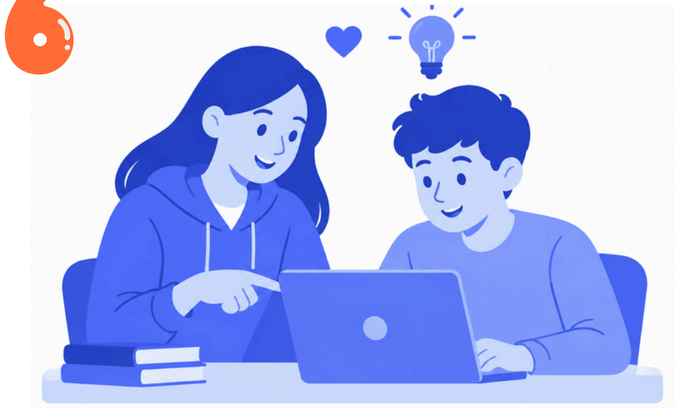
장비 · 도서 대여 관리

5



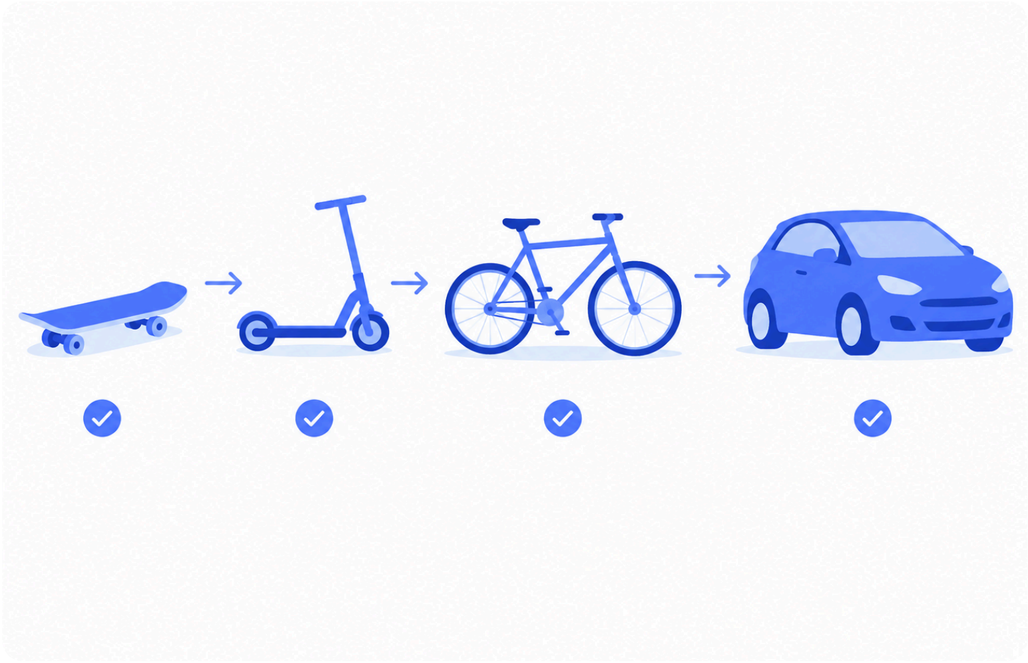
QR 출석 체크

6



선배 멘토 찾기

기획 2단계: 문제 → MVP



① 문제 정의

누가 · 언제 · 무엇이 불편한가?

② MVP 정하기

꼭 필요한 기능 딱 3개

작게 만들고 → 점점 키우기

[실습] 1페이지 기획서 만들기

💬 Antigravity 채팅창에 입력

동아리 신입 회원을 위한 서비스
'[내 아이디어]'의 기획서를
PLAN.md 파일로 만들어줘.

- 문제: 신입이 겪는 불편 1가지
- 대상: 누가, 언제 쓰는지
- 핵심 기능(MVP): 딱 3개
- 화면: 필요한 페이지 목록
- 기술 스택: 추천과 이유

실습 순서

① 새 폴더 club-helper 만들기
② 왼쪽 프롬프트 입력 (아이디어만 바꾸기)
③ PLAN.md를 읽고 말로 다듬기

- ⏱️ 약 10분 소요
- 기능이 3개를 넘으면 과감히 빼기
- 완성한 PLAN.md는 8차시에 사용!

7차시 → 8차시 진행 계획

오늘 (7차시)

주제 고르기 · 1페이지 기획서

PLAN.md 저장해 두기

다음 시간 (8차시)

바이브코딩으로 MVP 구현

한 명씩 예비 발표 · 피드백

질문과 답변

궁금한 점, 무엇이든 물어보세요!

다음 시간: 8차시 미니 프로젝트 구현 · 발표

기획서(PLAN.md)를 꼭 저장해 두세요.
8차시에는 그 기획서로 바로 구현을 시작합니다!

THANK YOU!!

8차시에 만나요~!

기획서(PLAN.md) 꼭 저장해 오세요!

해달 교육운영진 허진수 | 문의: 단톡방 / zintn11@gmail.com